Model Comparisons

Today's agenda:

- Assignment 3
- Model comparisons

Assignment 3

Due October 31 before midnight (note: I’ve update this from the syllabus)

Posted Soon (I’m still looking for a variety of data and models for part 1)

Assignment 3

Part 1:

- I give you full models and data
- You make alternative models and decide which is best
- E.g., Richness ~ elevation + latitude + temperature

Part 2:

- Fit models different models to your focal data, decide which fits the best
- You'll have example models from class, your book, and part 1 to modify

Assignment 3

Questions?

Last time

Confidence intervals

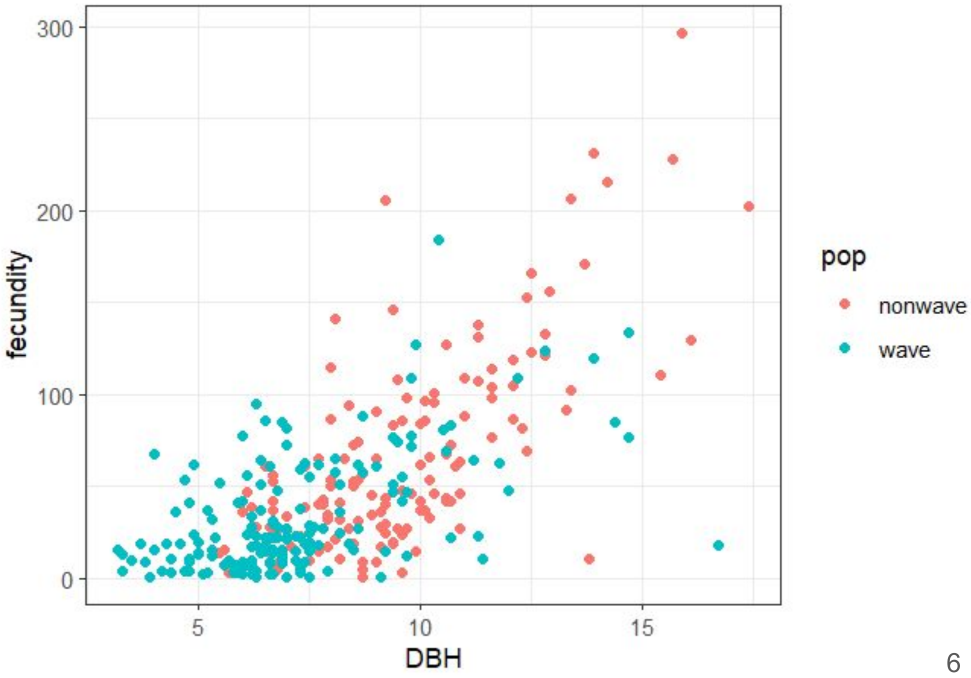
- What they are
- How to get them for a fitted function
- How to plot them

Today: how do we compare models?

Dataset: individual-level data from fir trees in New York

- Fecundity (cones produced)
- DBH (Diameter at breast-height)
- Population
    - Wave: Experienced wave-like die offs
    - Nonwave: Didn't

For more info, use `?Fir`



Modeling fecundity

Load the data using:

```
library(emdbook)

data("FirDBHFec_sum")
```

Make a quick plot of Fecundity vs DBH to look at the shape of the relationship

(optionally, color by population)

Modeling fecundity

We're going to model these data using:

$\mu = a*DBH^b$

Fecundity ~ NegBin(μ,k)

a,b,k are fitted parameters

Reminder: See Tables 3.1 and 4.1 for why we chose these functions

Modeling fecundity

μ = a*DBH^b

Fecundity ~ NegBin(μ,k)

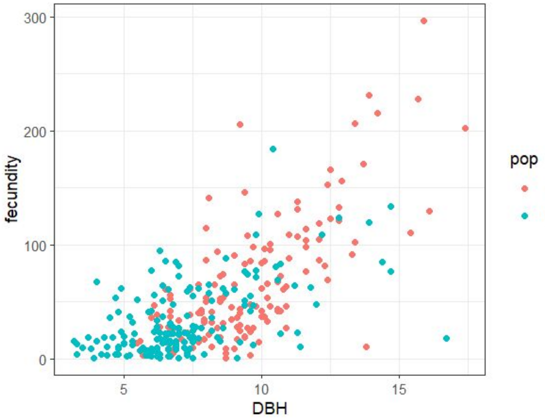
We've talked about model fitting and confidence intervals (CIs)

We can use this equation to ask:

- What is the most likely value of b?
- Does the CI of b include 1?
- How well does the model fit the data?
- How might selective logging impact seed numbers?

Modeling different populations

- But we have data on two populations
- We can modify the model to ask if and how they differ

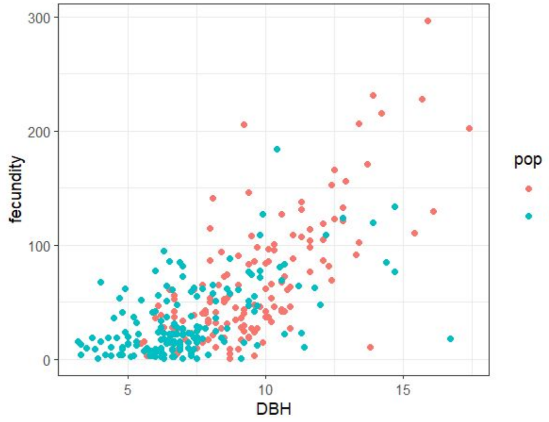


μ = a*DBH^b

Fecundity ~ NegBin(μ,k)

Modeling different populations

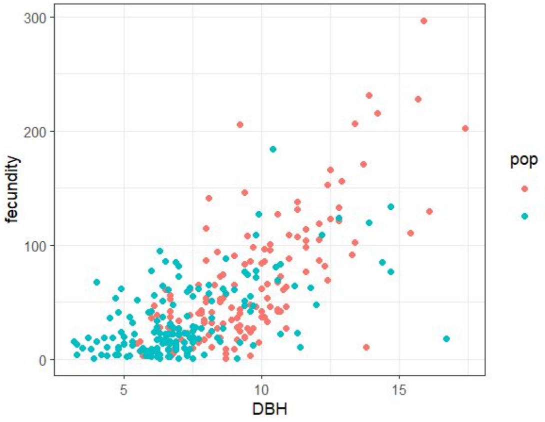
- But we have data on two populations
- We can modify the model to ask if and how they differ



$$\mu = a_i * DBH^{b_i}$$

$$\text{Fecundity} \sim \text{NegBin}(\mu, k_i)$$

Modeling different populations



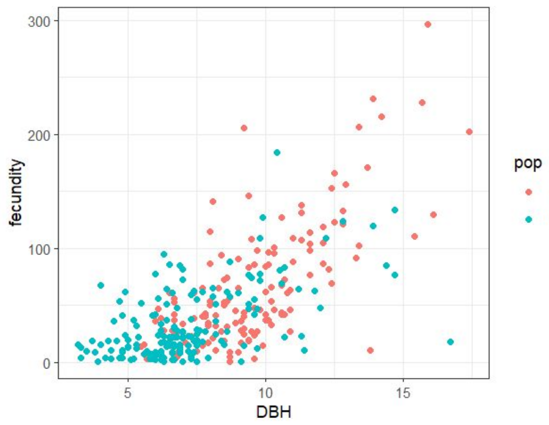
$\mu = a_i * DBH^{b_i}$

Fecundity ~ NegBin(μ,k_i)

We can now ask:

- Do different populations have different variance?
- Do different populations have different fecundity/DBH relationships
- Do the populations have the same baseline fecundity (*a*)?

Modeling different populations



$\mu = a_i * \mathrm{DBH}^{b_i}$

Fecundity ~ NegBin(μ,k_i)

We can now ask:

- Do different populations have different variance?
- Do different populations have different fecundity/DBH relationships
- Do the populations have the same baseline fecundity (*a*)?

We answer these questions with model comparisons

Model comparisons

We can use model comparisons for hypothesis testing

If there are population-specific differences,

adding population-specific terms should improve models, *e.g.*,

$$\mu = a_i * \mathrm{DBH}^{b_i} \quad \text{vs.} \quad \mu = a * \mathrm{DBH}^{b_i}$$

When should each model be preferred?

Likelihood Ratio Test

Last class we discussed the LRT as a way of finding confidence intervals

- LRT can also be used for model comparisons
- Only works for *nested* models

Nested models

Where the more complex model is a superset of the simpler model, *e.g.*,

Richness ~ latitude

Richness ~ latitude + elevation

Richness ~ latitude + elevation + time since glaciation

Richness ~ latitude + elevation + time since glaciation + distance to mainland

Nested models

Models that aren't nested might be:

Richness ~ latitude

Richness ~ elevation

Richness ~ time since glaciation

Richness ~ distance to mainland + clade age

Modelling our fir populations

```
library(emdbook)
library(bbmle)
data("FirDBHFec_sum")

FirDBHFec_sum$fecundity <- as.integer(FirDBHFec_sum$fecundity)
    # convert to integer to prevent errors

FirDBHFec_sum  <- na.omit(FirDBHFec_sum)
    # toss NA values (which cause errors)
```

Modelling our fir populations

Fit reduced model without population variation

Note this formulation requires less typing (didn't make a function)

```
nbfit.0 <- mle2(fecundity ~ dnbinom(mu = a * DBH^b,
                                    size = k),
                start = list(a = 1, b = 1, k = 1),
                data = FirDBHFec_sum)
```

Modelling our fir populations

Specify that we want separate “a” terms:

```
nbfit.a <- mle2(fecundity ~ dnbinom(mu = a * DBH^b,
                                     size = k),
                start = list(a = 1, b = 1, k =1),
                data = FirDBHFec_sum,
                parameters = list(a ~ pop))
```

Modelling our fir populations

Specify that we want separate “a” AND “b” terms:

```
nbfit.ab <- mle2(fecundity ~ dnbinom(mu = a * DBH^b,
                                      size = k),
                 start = list(a = 1, b = 1, k =1),
                 data = FirDBHFec_sum,
                 parameters = list(a ~ pop, b ~ pop))
```

Comparing models with LRT

In R, the anova() function will automatically conduct and LRT on mle2 models

- Anova: analysis of variance (we'll talk about this more later)

```
anova(nbfit.0, nbfit.a, nbfit.ab)
```

Run this now and we'll walk through the output

Comparing models with LRT

```
Likelihood Ratio Tests

Model 1: nbfit.0, fecundity~dnbinom(mu=a*DBH^b,size=k)
Model 2: nbfit.a, fecundity~dnbinom(mu=a*DBH^b,size=k): a~pop
Model 3: nbfit.ab, fecundity~dnbinom(mu=a*DBH^b,size=k): a~pop, b~pop

  Tot Df Deviance  Chisq Df Pr(>Chisq)
1      3   2891.0
2      4   2889.1 1.8526  1    0.17348
3      5   2883.5 5.6423  1    0.01753 *
---

Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Limits to LRT

A big drawback of LRT is the need for nested models

What if we want to compare a models that aren't nested?

E.g., a ~ pop vs b ~ pop

Then we need a different method

AIC

Akaike Information Criterion (AIC)

$$AIC = -2L + 2k$$

Where:

L = log-likelihood

k = number of model parameters estimates

AIC

AIC = $-2L + 2k$

Lower AIC = better model

AIC essentially penalizes your model for adding more parameters

Improvement to likelihood has to justify the additional parameters

Calculating AIC

In R we can use the function AIC() to get the AIC value for fitted models

Try it out!

```
AIC(nbfit.0)
```

Calculating AIC

The bbmle package has a really useful way of displaying AIC:

```
AICtab(nbfit.0, nbfit.a, nbfit.ab)
```

Run this and we'll walk through the output together

Calculating AIC

```
          dAIC df
nbfit.ab  0.0  5
nbfit.0   3.5  3
nbfit.a   3.6  4
```

Calculating AIC for other models

Now that we can compare non-nested models, let's try out a few more options

- Let's try a model (nbfit.b) where *b* differs by population
- Let's try a model (nbfit.abk) where *a*, *b*, and *k* differ by population

Modify the code we used for nbfit.ab to fit these new models

Then use `AICtab()` to compare these new models with the old ones

Which model(s) are best?

Why do we need to penalize AIC for more parameters?

Let's do a little experiment to check

Making a toy model

```
test_function <- function(x,a,b,sd){
     mu <- a+b*x
     out <- rnorm(n = length(x),mean = mu,sd = sd)
}

x_vec <-  runif(n = 1000, min = 0, max = 100)
y_vec <- test_function(x = x_vec, a = 1, b = 2, sd = 100)
```

Go ahead and plot y_vec vs x_vec

Adding predictors

Now we're going to make up some data that are entirely independent
(pick any values you like for these)

```
nonsense_predictor1 <- rnorm(n = length(x_vec),mean = 1000,sd = 2)
nonsense_predictor2 <- rbinom(n = length(x_vec),size = 1000,prob = .3)
nonsense_predictor3 <- rlnorm(n = length(x_vec),meanlog = .1,sdlog = .24)
```

Now, plot your y_vec against each of these nonsense predictors

Adding predictors

Now, we'll compare models with and without these nonsense predictors:

```
v0 <- lm(y_vec ~ x_vec) |> summary()

v1 <- lm(y_vec ~ x_vec + nonsense_predictor1) |> summary()

v2 <- lm(y_vec ~ x_vec+nonsense_predictor1 + nonsense_predictor2) |>
    summary()

v3 <- lm(y_vec ~ x_vec+nonsense_predictor1 + nonsense_predictor2 +
    nonsense_predictor3) |> summary()
```

Adding predictors

The R^2 of each mode is a measure of goodness of fit.

Let's compare the R^2 of all of these models

```
v0$r.squared

v1$r.squared

v2$r.squared

v3$r.squared
```

Which model performed the best for your? The worst?

Adding predictors

Adding more predictors improves model fit

even when the predictors aren’t meaningful!

So, we need other criteria to identify which models are meaningfully better

Before next class:

- Read 7.1 and 7.2